



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unidad 01 — Python básico

Bloque 04

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea



La idea en una frase

“

Una **clase** es un plano para crear **objetos** —con atributos para los datos y métodos para su comportamiento— y, como en Python todo es un objeto, crear tus propias clases es donde el lenguaje brilla de verdad.



Definir una clase

```
class Humano:
    # Atributo de clase (compartido por todas las instancias)
    especie = "H. sapiens"

    # Constructor: inicializa los atributos de cada objeto
    def __init__(self, nombre):
        # Atributo de instancia (propio e independiente de cada objeto)
        self.nombre = nombre

    # Método de instancia: opera sobre los datos del objeto actual
    def decir(self, msg):
        return f"{self.nombre}: {msg}"

    # Método de clase (compartido, recibe la clase como primer argumento)
    @classmethod
    def get_especie(cls):
        return cls.especie

    # Método estático (no recibe ni instancia ni clase)
    @staticmethod
    def roncar():
        return "*roncar*"

```



Tipos de métodos: Instancia, Clase y Estático

Tipo de Método	Decorador	Primer Parámetro	Propósito y Uso
De Instancia	Ninguno	<code>self</code>	Accede y modifica los atributos del objeto concreto. Se invoca desde la instancia (ej. <code>obj.decir()</code>).
De Clase	<code>@classmethod</code>	<code>cls</code>	Opera sobre la clase en general y sus atributos compartidos. No requiere instanciar la clase.
Estático	<code>@staticmethod</code>	Ninguno	Función utilitaria integrada en la clase por convención. No accede ni a <code>self</code> ni a <code>cls</code> .



`self` es la referencia al objeto actual dentro de un método. Permite acceder a sus propios atributos y métodos.



Creación de objetos y uso de métodos

```
# 1. Instanciar objetos
i = Humano(nombre="Ian")
j = Humano("Joel")

# 2. Uso de métodos de instancia (self apunta automáticamente a cada objeto)
print(i.decir("hola")) # "Ian: hola"
print(j.decir("hey")) # "Joel: hey"

# 3. Métodos de clase y modificación de atributos compartidos
print(i.get_especie()) # "H. sapiens"
Humano.especie = "H. neanderthalensis" # Cambia la especie para TODAS las instancias
print(i.get_especie()) # "H. neanderthalensis"
print(j.get_especie()) # "H. neanderthalensis"

# 4. Métodos estáticos (llamada directa desde la clase sin instanciar)
print(Humano.roncar()) # "*roncar*"
```



`self` representa el objeto actual. Python lo proporciona automáticamente al llamar a un método desde una instancia.



Constructor `__init__` y atributos por defecto

El método `__init__` es el constructor que se ejecuta automáticamente al crear un objeto:

```
class Coche:
    # Constructor con valores por defecto en los parámetros
    def __init__(self, marca, modelo, ano=2024):
        self.marca = marca
        self.modelo = modelo
        self.ano = ano

mi_coche = Coche("Seat", "León", 2022) # todos los parámetros
print(f"Coche 1: {mi_coche.marca} {mi_coche.modelo} ({mi_coche.ano})")

coche_nuevo = Coche("Seat", "Ateca") # usa el valor por defecto
print(f"Coche 2: {coche_nuevo.marca} {coche_nuevo.modelo} ({coche_nuevo.ano})")
```



Los parámetros con valor por defecto hacen la clase más cómoda sin perder configuración.



Representación de objetos: `__str__` vs `__repr__`

Los métodos especiales (o *dunder methods*) definen cómo se visualiza un objeto:

Método	Se activa con	Objetivo
<code>__str__(self)</code>	<code>print(obj)</code> , <code>str(obj)</code>	Representación limpia para el usuario.
<code>__repr__(self)</code>	<code>repr(obj)</code> , consola	Representación técnica, para depuración.

```
class Persona:
    def __str__(self):
        return f"{self.nombre}, {self.edad} años"

    def __repr__(self):
        return f"Persona('{self.nombre}', {self.edad})"

p = Persona("Ana", 25)
print(p)          # "Ana, 25 años"      (usa __str__)
print(repr(p))    # "Persona('Ana', 25)" (usa __repr__)
```



Herencia y polimorfismo

Una clase hija hereda atributos y métodos de una clase padre, para reutilizar y extender la lógica:

```
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre

    def hablar(self):
        raise NotImplementedError("La subclase debe implementar hablar()")

class Perro(Animal):
    def hablar(self):
        return "Guau"

class Gato(Animal):
    def hablar(self):
        return "Miau"

perro = Perro("Toby")
gato = Gato("Misi")
print(f"{perro.nombre} dice: {perro.hablar()}") # Toby dice: Guau
print(f"{gato.nombre} dice: {gato.hablar()}") # Misi dice: Miau
```



Las subclases reutilizan la estructura de la base y pueden redefinir comportamientos: eso es polimorfismo.



Mini-chequeo — POO

1. ¿Cuál es la diferencia entre un atributo de clase y uno de instancia?
2. ¿Qué hace `self` en un método de instancia?
3. ¿Cuándo se usa `@staticmethod` en lugar de un método normal?

Respuestas:

1. El atributo de clase es compartido por todos los objetos de la misma clase. El de instancia es propio de cada objeto.
2. `self` es la referencia al objeto actual. Permite acceder a sus atributos y métodos dentro de la clase.
3. Cuando el método no necesita acceder a `self` ni a `cls`: es una función utilitaria que "vive" dentro de la clase por convención.



Módulos: Definición e importación

Los módulos son archivos `.py` con código reutilizable. Python trae cientos en su biblioteca estándar y `pip` te permite instalar miles más.

Sintaxis	Ejemplo	Ventaja
<code>import modulo</code>	<code>import math</code> → <code>math.sqrt(16)</code>	Espacio de nombres aislado (evita colisiones).
<code>from modulo import x</code>	<code>from math import ceil</code> → <code>ceil(3.7)</code>	Importas solo lo necesario y lo usas directo.
<code>import modulo as alias</code>	<code>import math as m</code> → <code>m.sqrt(16)</code>	Abrevia nombres largos.
<code>from modulo import *</code>	<code>from math import *</code>	⚠ No recomendable: puede sobrescribir nombres.

```
import math
print(math.sqrt(16))    # ⇒ 4.0
print(dir(math))        # Lista todas las funciones del módulo
```



Usa `dir(math)` para listar atributos y `help(math.sqrt)` para su documentación.



Gestor de paquetes `pip` y dependencias

pip descarga e instala paquetes desde **PyPI** (Python Package Index), el repositorio oficial con más de 400.000 paquetes.

```
pip install requests          # Instala la última versión
pip install requests==2.28.0  # Instala una versión exacta
pip list                      # Paquetes instalados
pip uninstall requests        # Elimina un paquete
```

El archivo `requirements.txt`

Guarda la lista de dependencias para asegurar la reproducibilidad:

```
pip freeze > requirements.txt  # Generar con las versiones actuales
pip install -r requirements.txt # Recrear el entorno en otro equipo
```



Conserva las versiones para que el proyecto se reproduzca en cualquier equipo.

Generadores y la palabra clave `yield`

Crean valores **sobre la marcha** (*lazy evaluation*) en lugar de cargarlos enteros en memoria. Ideales para secuencias grandes:

```
# Función generadora (contiene yield)
def duplicar_numeros(iterable):
    for i in iterable:
        yield i + i # Produce un valor y pausa hasta la siguiente petición

for i in duplicar_numeros(range(1, 900000000)):
    print(i)
    if i >= 30:
        break # Se interrumpe sin generar los 900 millones de elementos
```

Generador vs lista

Estructura	Sintaxis	Memoria
Comprensión de lista	<code>[x ** 2 for x in range(1000000)]</code>	Alta: el millón de elementos en RAM a la vez.
Expresión generadora	<code>(x ** 2 for x in range(1000000))</code>	Mínima: genera de uno en uno al pedirlos.

 La diferencia es el operador: `[]` crea una lista, `()` crea un generador.



Decoradores y @wraps

Un **decorador** es una función que envuelve a otra para añadirle comportamiento sin modificar la original.

```
from functools import wraps

def mi_decorador(funcion):
    @wraps(funcion) # Preserva los metadatos (__name__, __doc__)
    def wrapper(*args, **kwargs):
        print("Antes de llamar a la función")
        resultado = funcion(*args, **kwargs) # Ejecuta la original
        print("Después de llamar a la función")
        return resultado
    return wrapper

@mi_decorador
def saludar(nombre):
    print(f"Hola, {nombre}")

saludar("Ana")
# Antes de llamar a la función
# Hola, Ana
# Después de llamar a la función
```



Decoradores avanzados e integrados

Decorador con parámetros

```
def pedir(funcion):  
    @wraps(funcion)  
    def wrapper(*args, **kwargs):  
        mensaje, decir_por_favor = funcion(*args, **kwargs)  
        return f"{mensaje} ¡Por favor!" if decir_por_favor else mensaje  
    return wrapper  
  
@pedir  
def say(decir_por_favor=False):  
    return "¿Me compraste una cerveza?", decir_por_favor  
  
print(say()) # "¿Me compraste una cerveza?"  
print(say(decir_por_favor=True)) # "¿Me compraste una cerveza? ¡Por favor!"
```



Úsalos cuando la abstracción aporte claridad: registro, permisos, validación o medición.

class: sparse-slide

Decoradores integrados habituales

Ya los has usado en la unidad de POO

Decorador	Función
<code>@classmethod</code>	Método de clase (recibe <code>cls</code>).
<code>@staticmethod</code>	Método estático (sin <code>self</code> ni <code>cls</code>).
<code>@property</code>	Método como propiedad de solo lectura (getter).
<code>@wraps</code>	Conserva nombre y documentación de la función decorada.



Mini-chequeo — Módulos y avanzado

1. ¿Qué diferencia hay entre `import math` y `from math import sqrt`?
2. ¿Cuándo es mejor usar un generador que una lista?
3. ¿Qué hace un decorador a una función?

Respuestas:

1. Con `import math` importas todo el módulo y accedes con `math.sqrt()`. Con `from math import sqrt` importas solo esa función y la usas directamente como `sqrt()`.
2. Cuando la secuencia es grande o no necesitas todos los valores a la vez. El generador ocupa mucho menos memoria.
3. La envuelve y añade código antes y después de ejecutarla, sin modificar la función original.



Resumen — Bloque 04

- Una clase es un plano; un objeto es una instancia de ese plano.
- `__init__` inicializa los atributos; `self` referencia al objeto actual.
- Los métodos de clase (`@classmethod`) operan sobre la clase; los estáticos (`@staticmethod`) no dependen de nada.
- Los módulos se importan con `import` ; `pip` instala paquetes externos desde PyPI.
- Los generadores (`yield`) crean valores sobre la marcha, ahorrando memoria.
- Los decoradores envuelven funciones para añadirles comportamiento extra.



Vocabulario rápido

Término	Idea general
Clase	Plano para crear objetos
Objeto/Instancia	Ejemplo concreto de una clase
Atributo	Dato que pertenece a un objeto
Método	Función que pertenece a una clase
<code>self</code>	Referencia al objeto actual
Herencia	La clase hija hereda de la padre
<code>__init__</code>	Constructor, se ejecuta al crear
<code>@classmethod</code>	Opera sobre la clase (<code>cls</code>)
<code>@staticmethod</code>	Sin <code>self</code> ni <code>cls</code>

Término	Idea general
Módulo	Archivo <code>.py</code> reutilizable
pip	Gestor de paquetes de Python
PyPI	Repositorio oficial de paquetes
requirements.txt	Lista de dependencias
Generador / yield	Función que produce valores uno a uno
Decorador	Envuelve otra función para ampliarla
@wraps	Preserva los metadatos de la original

